ORIGIN — Distribution Intelligence & Inventory Optimization Engine

A Research-Backed Technical Project Plan for AI-Assisted FMCG Distribution, Inventory Allocation, and Waste Reduction

TEAM NAME: ORIGIN
TEAM LEADER: ASHALAN SYED
PROJECT CATEGORY: AI-Powered Supply Chain and Distribution Optimization
PROBLEM STATEMENT NUMBER: 2

Executive Summary

In FMCG distribution, mismatches between supply and demand lead to stockouts, lost sales, excess inventory, and waste. For example, one warehouse may hold surplus juice bottles while a nearby retailer runs out, forcing lost sales and emergency transport. ORIGIN proposes a **web-based decision-support engine** that analyzes inventory, forecast demand, lead times, transport costs, and expiries to generate ranked, explainable transfer recommendations across the network. The system will use transparent calculations (simple forecast averages, days-of-cover, safe surplus, and gap metrics) to highlight urgent needs and feasible transfers. Unlike existing tools (e.g. Zoho Inventory or Odoo, which handle stock tracking and reorder levels) or heavyweight solutions (e.g. SAP IBP or Blue Yonder, which require extensive data and complex models), ORIGIN focuses on a targeted workflow: **jointly evaluating supply surpluses and destination shortages to reduce stockouts, costs, and waste.** We will implement a functional prototype with CSV data input, an interactive dashboard, and comparison against a baseline (static policy). Performance will be measured on stockout risk, fill rate, and costs. All claims and design decisions are backed by industry sources or clearly labeled assumptions.

Key Findings: Verified data show global supply inefficiencies: about one-third of food is wasted (1.3B tons/year)【34†L13-L21】, India's logistics costs are ~8% of GDP (2024)【45†L99-L107】, and retail stockouts average ~8% of demand【43†L31-L39】 (causing ~$15–20B lost sales/year in US food alone 【43†L7-L9】). Major platforms (Zoho Inventory, Odoo, SAP IBP, Blue Yonder, Kinaxis) support multi-warehouse tracking, reorder rules, demand forecasting, and optimization, but none explicitly combine all factors (surplus, demand gap, cost, expiry) into a single ranked action list with transparent reasoning. ORIGIN's hypothesis is that a focused, explainable tool can bridge this gap for mid-market FMCG use-cases.

1. Original Problem Statement

Problem Statement 2: Distribution Intelligence & Inventory Optimization Engine. FMCG companies use multiple warehouses and distributors feeding retail stores. Regional demand variability, stock levels, transport costs, and product shelf-life make distribution complex. Imbalances cause stockouts at some locations and excess at others, hurting sales and raising costs. The solution must analyze inventory, demand, and constraints to recommend *what products to move, to which destinations, how many units, and from which source*, so as to reduce stockout risk, unneeded transport, and waste. The outcome is a software tool that gives actionable distribution recommendations. Real or simulated data may be used, with assumptions stated.

This aligns with the challenge: improve end-to-end distribution by aligning supply to demand. Our prototype will explicitly address each requirement: it will track inventory at multiple sites, consider forecast demand and shelf-life, and output allocation suggestions (product, quantity, source, target). We will ensure that **each bullet of the hackathon statement is met**, as detailed later (Sect. 4).

2. Problem in Real-World Terms

Consider an FMCG company selling bottled juice. Suppose **Warehouse A** in the city has 200 bottles of Apple juice with expiry 6 months away, while **Warehouse B** in the suburbs has only 30 bottles. Retailer **North** near Warehouse A has high daily demand (50 bottles/day) but currently only 10 in stock. Retailer **South**, far from North, has low demand (10/day) but holds 50 bottles. If managers ignore the network context, they might not transfer stock.

- **Stockout:** North runs out of Apple juice: customers leave empty-handed, causing lost sales and goodwill.
- **Overstock:** South has 40 bottles more than needed; these may tie up capital or expire unsold.
- **Inventory imbalance:** Plenty of stock exists in the network (in A and South) but not where needed (North).
- **Expiry risk:** If the bottles near expiry remain idle, they must be discarded (waste).
- **Demand forecasting:** We estimate North's demand (e.g. 50/day average) from past sales data.
- **Inventory optimization:** We decide how much each location should hold or send.
- **Distribution optimization:** We choose a source (A or South), destination (North or others), quantity, and transport route (considering costs and lead-times).

These issues are interrelated: a stockout at North might have been prevented by earlier transfer from A, but that requires forecasting and knowing A had safe surplus. If done separately, warehouse A might not have ordered replacements in time, or might keep shipping to South (even though it doesn't need more). A true distribution optimization views all warehouses and retailers together, balancing shortages and surpluses.

Connection of issues: A forecast suggests North needs 50/day. Without considering Warehouse A's surplus, North may stay out-of-stock. Without considering South's low demand, you wouldn't reallocate from South to North. If transport cost or time from A is high, maybe South (if it had surplus) could supply faster. Ignoring expiry, you might send nearly-expired bottles to a distant store (arriving too late). The problem requires joint demand forecast, stock status, transportation, and expiry data.

Scope: The hackathon problem specifically covers analyzing data to generate allocation recommendations. We do *not* promise to build full ERP integration, real-time tracking, or on-the-ground execution. We also distinguish:

- *Hackathon problem:* generically "optimize distribution of FMCG across warehouses and stores."
- *Wider industry:* Many resources (e.g. FMCG supply chain studies) highlight billions lost to stockouts and waste【43†L7-L9】【34†L13-L21】. Our solution targets a slice of this problem: an *explainable allocation engine*, not a complete logistics automation system.
- *Prototype's focus:* We will show how to translate data (inventory, sales history, routes) into a prioritized action list. We explicitly **won't** reinvent basic WMS (as Zoho/Odoo already track stock) nor claim to surpass enterprise suites on all fronts. We focus on *combining* data points (demand, expiry, costs) to rank transfers.

(At least one visual example here: stock in A vs shortage in North vs overstock in South.)

3. Real-World Impact and Evidence

Inefficiencies in FMCG distribution have a large economic impact. We gather evidence on: supply-chain costs, stockouts, waste, carrying costs, and forecasting gaps.

Metric	Verified Figure	Geography & Year	Source	Relevance to ORIGIN	Limitations
Share of food lost/ wasted	~30% (1.3 billion tons) of global food production (≈2011 data)	Global (FAO, 2011 study) 【34†L13-L21】	FAO (2012), *Global food losses and food waste*	Highlights scale of edible product wastage; implies potential FMCG expiry waste if not allocated	Includes losses at farm/ processing, not just retail expiries; older data
Logistics cost of economy	~7.97% of GDP (₹24 lakh crore ≈$272 bn) (FY2023–24)	India (2023–24) 【45†L99-L107】	Govt. of India report via PIB 【45†L99-L107】	Confirms high transport/ warehouse cost burden in India's supply chains	Covers all logistics (all sectors), not only FMCG; progressive improvement in data reliability
Retail out-of-stock rate	~8% of items (global retail, early 2000s)	Global (Retail industry avg, early 2000s) 【43†L31-L39】	Oracle NetSuite (citing industry data)【43†L31-L39】	Shows typical frequency of stockouts; implies potential sales lost	Old data; likely worsened by recent disruptions (COVID-19)
Retail food lost sales	$15–20 bn per year (≈3% of sales)	USA, retail food (recent estimate) 【43†L7-L9】	Oracle NetSuite guide【43†L7-L9】	Quantifies direct revenue loss from stockouts in food retail	Specific to US food retail; may not scale to all FMCG or India
Inventory carrying cost	**20–30% of inventory value per year** (goal)	Industry practice (general guideline) 【51†L799-L807】	Fishbowl blog (citing ISM) 【51†L799-L807】	Indicates cost of holding excess stock; motivates reducing overstock	Not an official stat; no single standard benchmark; range is broad guideline

Metric	Verified Figure	Geography & Year	Source	Relevance to ORIGIN	Limitations
Logistics cost share (mode)	Road: ₹11.03/ tonne·km; Rail: ₹1.96/tonne·km; Water: ₹1.80–3.30/tonne·km (2023)	India (2023) 【44†L29-L37】	LinkedIn post citing *Assessment of Logistics Cost in India*【44†L26-L34】	Illustrates high road transport costs vs underused cheaper modes	Secondary source (LinkedIn); values context-specific

Why these matter: The global food waste stat【34†L13-L21】 shows that supply-chain losses are massive. In FMCG context, unsold perishables at expiry are a fraction of this, but even small percentages mean large volumes. India's logistics cost (~8% GDP)【45†L99-L107】 means efficiency gains are critical to competitiveness (the National Logistics Policy targets <10% GDP). High logistics costs make transport decisions crucial. Stockouts (8% of items)【43†L31-L39】 and associated ~$20B lost sales in US food retail【43†L7-L9】 underscore that unmet demand hurts revenue. Inventory carrying costs (20–30%)【51†L799-L807】 remind us that excess stock ties up capital. All these support ORIGIN's focus: to **shift stock proactively** to meet demand (avoiding stockouts) while not overstocking and incurring high holding costs or waste.

Limitations: Some figures (e.g. FAO food waste) include losses at farming and processing. FMCG packaged goods waste is smaller, but still significant. Our prototype won't claim to eliminate global waste, only to improve distribution decisions. The carry-cost figure is a general guideline, not precise, but illustrates that inventory has large financial drag. We did not find an exact India-specific "FMCG inventory waste $" stat; such granular data seems unavailable. All statistics are from trusted sources (FAO, government reports, industry analyses) to ground the problem scale.

4. Existing Solutions and Competitors

We review relevant inventory/distribution platforms to see how they address multi-location stock management and planning. We focus on officially documented features. The table below compares each solution's capabilities with ORIGIN's goals.

Company / Product	Verified Capabilities (with sources)	Overlap with ORIGIN	ORIGIN's Proposed Focus	Evidence & Notes
Zoho Inventory	*Multi-warehouse management:* "Manage your stock across multiple warehouse locations, transfer items within warehouses"【63†L235-L244】. *Batch/serial tracking:* "Track items based on batches and serial numbers"【64†L1-L4】. *Reorder Alerts:* low-stock reorder points. *Picklists, shipments, carrier integrations.*	- Basic inventory tracking across warehouses. - Simple stock transfer orders. - Alerts on reorder levels.	- Algorithmic ranking of transfers vs. static reorder. - Explicit cost/lead-time/expiry analysis.	Zoho is an online inventory/WMS for SMEs. It supports batch expiry info (implied by "track by batches"【64†L1-L4】) and can alert reorder needs. It does **not** natively compute optimized allocations or factors like delivery time. ORIGIN will simulate Zoho's inputs (multi-location stock, demand) but focus on decision logic.
Odoo Inventory	*Automated replenishment:* Min/max rules ("reorder points" and quantities), MTO, Master Production Schedule【77†L134-L142】. *Multi-locations:* handles multiple warehouses and inter-location transfers【77†L134-L142】. *Perishables:* expiration dates, FEFO picking【77†L144-L146】. *Bills of materials (for assembly).*	- Automated reorder based on rules. - Location and lot-tracking (expiry). - Inventory moves.	- Joint evaluation of all routes/ sources (beyond single-warehouse reorder). - Ranking by urgency/cost (not just rule threshold).	Odoo is an open-source ERP. Its Inventory app can set *reordering rules* and multi-warehouse moves; a partner site confirms "Manage multiple warehouses… advanced reordering based on forecast/lead times"【77†L134-L142】. It supports expiry/lot tracking【77†L144-L146】. However, it triggers individual replenishment (e.g. auto-create PO) rather than producing a combined ranked plan. ORIGIN fills this gap by synthesizing multiple replenishment options into a prioritized list with explanations.

Company / Product	Verified Capabilities (with sources)	Overlap with ORIGIN	ORIGIN's Proposed Focus	Evidence & Notes
SAP Integrated Business Planning (IBP)	*Modules:* Demand management (AI forecasts), Inventory planning, Response & supply planning, S&OP【98†L35-L44】【98†L208-L218】. *Functions:* Multilevel supply modeling, what-if simulations, inventory optimization balancing cost vs. service【98†L208-L218】. *Data integration:* real-time analytics on SAP HANA.	- Holistic planning across network. - Forecast and supply matching with optimization. - Can consider lead times and inventory targets.	- Interactive demo scope: simplified network focus. - Emphasis on transparency/ explanation (vs. IBP "black-box"). - Designed for smaller data (CSV input) not full ERP data.	SAP IBP is an enterprise cloud SCM suite. It indeed handles demand and supply planning, multi-echelon inventory optimization, and S&OP on one platform【98†L35-L44】【98†L208-L218】. It uses advanced algorithms. IBP overlaps on goals (balancing stockouts vs. cost) but is heavyweight. ORIGIN won't match IBP's scale or performance, but aims for an explainable prototype. We assume IBP supports shipping decisions but have no doc quote on explicit allocation features, so ORIGIN's direct recommendations (with cost detail) would be a point of differentiation.

Company / Product	Verified Capabilities (with sources)	Overlap with ORIGIN	ORIGIN's Proposed Focus	Evidence & Notes
Blue Yonder (Luminate)	*Supply chain planning:* demand planning, inventory optimization, allocation & replenishment【88†L72-L82】. *Scenario planning, AI insights* (marketed). *Logistics/TMS:* network design, transportation optimization (Luminate Logistics). *Omni-channel order management.*	- Advanced replenishment and allocation planning. - Multi-echelon inventory forecasting and optimization.	- Explainable line-by-line recommendations (vs. automated AI suggestions). - Lightweight prototype interface (compared to Yonder's suite).	Blue Yonder (formerly JDA) offers the Luminate platform. A review notes it includes *"Demand planning, Inventory optimization, Allocation & replenishment"*【88†L72-L82】. It's aimed at large retailers. We cannot probe its internals, but it's known for AI-driven forecasts and optimizations. We learn from it that end-to-end planning exists. ORIGIN will not attempt Luminate's scale or AI sophistication; instead, it shows the concept on a smaller dataset with clear logic.
Kinaxis RapidResponse	*Concurrent planning:* demand, supply, inventory, S&OP (Holistic SCM)【95†L151-L159】. *Scenario modeling:* near real-time what-if analysis, decision modeling. *Connected data model* for multi-tier chains.	- Integrated planning across functions (demand, supply, inventory). - Scenario-driven decision support.	- Focus on specific distribution scenario (transfer list) rather than full concurrent simulation. - Transparent rules vs. Kinaxis agents.	Kinaxis (Maestro) is a planning orchestrator for complex supply chains【95†L151-L159】. It emphasizes simultaneous updates across demand/supply/inventory. While it can model transfers, its strength is scenario analysis. ORIGIN will use a simpler sequential decision flow, not real-time concurrency. We do not have a direct feature list, but Kinaxis's scope suggests ORIGIN targets a narrower subset (operational allocation) in a more interpretable way.

5. Differentiation and Workflow Gap Analysis

Most existing tools manage inventory or planning **individually**. We identify potential gaps:

- **ERP/WMS inventory modules (e.g. Zoho/Odoo):** Strength: accurate stock records, basic reorder alerts【63†L235-L244】【77†L134-L142】. Gap: do not optimize network-wide moves. ORIGIN: will rank source-destination pairs by urgency, cost, expiry, etc.

- **Advanced supply-chain planners (SAP IBP, Blue Yonder):** Strength: powerful forecasting and multi-echelon optimization【98†L35-L44】【88†L72-L82】. Gap: typically complex, expensive, and opaque; may not show "why" decisions are made. ORIGIN: trades scale for explainability; hypotheses: a simpler algorithm and UI can still improve decisions for small-to-mid-scale users, by being transparent and focus on immediate transfers.

- **Manual/heuristic processes:** Strength: human oversight. Gap: slow, reactive, prone to silo decisions. ORIGIN: automates and makes data-driven recommendations.

Our **hypothesis**: No current product explicitly takes *inventory* + *demand* + *expiry* + *cost* + *lead time* together to *rank* individual transfer actions with explanations. This is not proven – it's a gap we will test. Table below outlines:

Existing Approach	Documented Strengths	Potential Gap / Limitations	ORIGIN's Testable Improvement
Basic WMS/ ERP (Zoho/ Odoo)	Provides multi-location inventory visibility and basic replenishment rules【63†L235-L244】【77†L134-L142】. Automated order creation by min/max levels.	Lacks holistic cross-location allocation logic. Does not rank by urgency or cost; no expiry-aware suggestions.	Supply-demand balancing via transparent formulas: e.g. link surplus vs. shortage directly, include cost/expiry filters. ORIGIN will test if this reduces unmet demand without excess shipments.
Advanced Planning Suites (SAP, Blue Yonder, Kinaxis)	Comprehensive optimization (multi-echelon, AI-driven)【98†L35-L44】【88†L72-L82】; strong scenario planning.	Complex setup; cost prohibitive for small teams; opaque outputs. Unclear if they present line-item actionable recommendations.	A focused rule-based engine (no big ML) may achieve significant benefit on medium-scale cases. ORIGIN will compare baseline rules vs. its ranked list on stockout/cost metrics to verify benefit.
Manual/ static policy	Human judgment; no system cost.	Slow, no data integration; often ignores transport/expiry in real-time decisions.	Automated "expert system" approach: generate prioritized recommendations faster and with data consistency. We will compare static thresholds (e.g. restock at safety stock) against ORIGIN's dynamic results.

In summary, our *design hypothesis* is that even if general SCM systems cover pieces of this problem, an **explainable allocation engine** can be a novel assistance tool. We do not claim it's inherently "better" (that requires testing), but we will build an MVP and compare it against simple baselines to quantify its added value.

6. Prototype Format Choice

We will build a **web application** as the demonstrator. A browser-based app allows judges and users to: upload data easily (via CSV), interact with filters and charts, and see multiple screens side-by-side. Web UIs can display rich dashboards (warehouse network maps, tables, plots) and detailed recommendation lists. It also fits easily on laptops and can be shown live without installation. For example, the **Dashboard** can summarize metrics (total network inventory, urgent items, costs). The **Recommendations** screen can allow interactive approval/inspection. A mobile app or only a chatbot/ prototype would not suitably present detailed tables or maps. A web app ensures an end-to-end workflow: data in, processing backend (Python), interactive frontend (React or similar), and export of results.

Thus, ORIGIN will be a **single web-based dashboard system** with multiple pages (screens) for data upload, network view, risk analysis, etc. This is practical for hackathon demo and aligns with industry UI (many supply-chain SaaS use web UIs).

7. Web Application Design

We design the app as a dashboard with the following screens:

- **Screen 1 — Overview Dashboard:** Shows key summary metrics. E.g.:
- *Total usable inventory* in the network.
- *Number of warehouses and retail locations*.
- *Number of SKU-location combos at high stockout risk*.
- *List of locations projected to stockout (and by when)*.
- *List of locations with surplus stock*.
- *Inventory near expiry* count (if batch data provided).
- *Pending recommended transfers* count.

- *Estimated current vs recommended transport cost*.
  These are computed from loaded data (or marked simulated). It orients the user to major issues.

- **Screen 2 — Data Upload & Validation:** Users upload CSV files (inventory, sales, warehouses, routes). The system checks for errors:

- Missing values (e.g. no stock listed),
- Duplicates (same SKU-location repeated),
- Invalid dates (sale before manufacturing date),
- Negative quantities,
- Inconsistent product IDs across files,
- Missing route/lead time between warehouses and destinations,

- Expired batches (inventory past expiry).
  Detected issues are listed with warnings. For example, "No route defined between WH-A and Retail-South: transfer cost cannot be computed." This informs users that some transfers cannot be planned due to data gaps. No automatic imputations are done.

- **Screen 3 — Inventory Network:** Visual map or schematic of locations (warehouses, distribution centers, retailers) and their connections. E.g., nodes with inventory levels and arrows showing available transport routes (annotated with lead times and costs). If geographic data is not real, a simple graph layout (nodes and edges) can suffice. The map highlights stock levels (maybe colored by fullness) and route time/cost.

- **Screen 4 — Risk Analysis:** Tabular or chart view per SKU-location:

- *Estimated daily demand*, *current usable stock*, *incoming shipments*, *days of coverage* (stock/demand), *projected shortage gap*, *needed-by date*, *days to expiry*.
  For example, "Retail North (SKU Apple Juice): demand 50/day, stock 10, incoming 0, cover 0.2 days, gap 40 units, need by 2026-11-05." Color warnings highlight urgent shortages. If demand data is sparse, flag confidence. This helps judge the urgency before transfer recommendations.

- **Screen 5 — Recommendation Engine:** Core interface. A ranked list of suggested transfers, each row showing:

- **Rank** (1,2,3...).
- *Product/SKU* name.
- *Source Warehouse* ID.
- *Destination Location* ID.
- *Quantity proposed*.
- *Estimated transport cost* (units × cost or number of trips × trip cost).
- *Delivery lead time*.
- *Destination's demand gap (units needed)*.
- *Source surplus before and after transfer*.
- *Destination stock after transfer (projected)*.
- *Constraints check:* which constraints were satisfied or causing rejections (e.g. "OK: matches replenishment gap, fails: vehicle capacity").

- *Explanation:* e.g. "Urgent: Retail North has 8 days' stock left vs need-by, Warehouse A has 60 safe surplus, cost is ₹X, shipping arrives in 2 days (before need date)."
  Each recommendation can be expanded or clicked for details. Actions like "Approve", "Reject", or "Export" could be shown (though in prototype these do not actually execute shipments). The idea is transparency: judges can see *why* each recommendation was made and what constraints were applied.

- **Screen 6 — Scenario Comparison:** Side-by-side comparison of "Baseline" vs "ORIGIN". Both scenarios start with the same initial state (inventory, demand, constraints). Baseline might be, say, "No transfers, or only standard reorder triggers". ORIGIN scenario applies the recommended transfers. Show metric outcomes: projected stockouts, fill rate, total cost, expiry leftover. For example, charts of "fulfilled vs demanded units" in each scenario. The goal is to show measurable improvement (e.g. fewer stockout days) in the ORIGIN case.

- **Screen 7 — Evidence & Assumptions:** Documentation page listing:

- Data sources (e.g. synthetic, public example data).
- Assumptions (e.g. constant daily demand model).
- Forecast method (e.g. moving average over last 7 days).
- Optimization rules (briefly, e.g. "Minimize stockouts priority, then cost").
- Known limitations (e.g. "Does not integrate sales price", "No real-time tracking").

- Test date and version.
  The idea is auditability: any judge or engineer should understand how the figures were derived and what is simulated vs. real.

(Each screen would ideally have diagrams or charts; these would be included in the final report as figures with the descriptions above.)

8. System Architecture

The system follows a modular pipeline:

```
[CSV Data Sources]
    → [Validation Module]
    → [Demand Estimation Module]
    → [Inventory Risk Analysis Module]
    → [Allocation Optimization Module]
    → [Explainable Recommendations]
    → [User Approval / Export]
    → [Baseline/ORIGIN Scenario Evaluator]
```

- **CSV Data Sources:** Inventory on-hand by SKU/location (with batch/expiry info), historic sales, list of warehouses, list of retail/other locations, transport routes with lead-times and unit costs. These are uploaded by user.

- **Data Validation:** Checks input quality (see Screen 2). Input: raw CSVs. Output: flagged issues and cleaned tables (missing rows left out, consistent formats). If critical data missing (e.g. no route), affected transfers are simply disallowed later.

- **Demand Estimation:** Input: sales history CSV for each SKU-location. Output: *Estimated daily demand* per SKU/location (or a demand forecast for next N days). Algorithm: e.g. compute 7-day moving average if $\geq$7 days exist; otherwise use available data with a warning. This output feeds risk analysis (for each dest) and also supply planning (deciding needs).

- **Inventory Risk Analysis:** Input: current on-hand stock, batch expiries, incoming shipments (if any), demand estimates. Calculates for each SKU-location: *Days of cover* = stock/demand; *Lead-time demand* = demand × lead-time; *Replenishment gap* = max(0, LeadTimeDemand + SafetyStock – (stock + incomingBeforeNeedDate)). It considers timing: only incoming stock before the projected need date counts. Also computes *expiry risk* (e.g. units expiring before sale). Output: risk metrics for each location (shortage gap, need-date, cover days).

- **Allocation Optimization:** Input: Surplus info (for each source SKU: safe surplus = stock – protected reserve), and each destination's gap, plus transport network data. The core algorithm loops over (SKU, dest) needing stock in order of urgency (smallest days-of-cover, largest gap). For each, find feasible source(s) with surplus. Check constraints: source-surplus, destination's capacity, route exists, lead-time $\leq$ needed time, batch life sufficient. Also consider transport cost. Compute a *transfer suggestion* = min(gap, source_surplus, capacity). Score each suggestion by a weighted formula (e.g. high priority for urgent shortages, then lower cost). Sort by score to rank recommendations. If a suggestion is infeasible, record reason. (The exact formula weights are design choices to be tested.)

- **Explainable Recommendations:** Each output record includes all inputs and checks (as in Screen 5), plus a textual reason. This is then presented in the UI.

- **User Approval / Export:** User can review and accept or reject each suggestion (prototype: just record rejections). Approved transfers could be exported as CSV or shipped to ERP in a real system; here we just simulate selection.

- **Baseline vs ORIGIN Scenario Evaluator:** Takes the initial inventory and simulates two scenarios over a planning horizon (e.g. next 14 days):

- *Baseline:* maybe no transfers, or transfers only when stock hits reserve (a simple rule).
- *ORIGIN:* apply recommended transfers now, then normal consumption.
  Calculate metrics: total stockouts, fill rate, transport costs, expired units, rule violations. This allows measuring ORIGIN's impact.

Technology choices: A fast setup is needed. We propose **React (or Streamlit)** for the frontend dashboard (flexible charts, easy development). Backend in **Python** (common for data tasks): using pandas for table ops. Storage can be simple CSV or SQLite. Visualization via a library like Chart.js or D3 via React, or Plotly/Altair. For optimization, a simple custom algorithm suffices; no need for heavyweight solvers (we can compare basic heuristics vs. OR-Tools if time allows, but transparency is key). This stack (Python + React) is common in startups and hackathons: easy integration with Web APIs (Flask/FastAPI if needed) and good community support. We avoid unnecessary complexity: no microservices or big frameworks, since it's a prototype. If team familiarity or time push suggests, Streamlit (Python-only, with built-in UI) could speed up development at the cost of less customization.

9. Algorithmic Logic and Formulas

We will use clear formulas:

- **Usable Inventory:**
  $$ \text{UsableStock} = \text{OnHand} - \text{Reserved} - \text{Expired} $$
  (We assume no complex reservations beyond a simple protected quantity.)

- **Estimated Daily Demand:**
  Using recent sales data: e.g.
  $$ D = \frac{\sum_{i=1}^n \text{Sales}_{i}}{n} $$
  For example, if last 7 days had 350 units sold, $D = 50$/day. If data is sparse, we note low confidence.

- **Days of Cover:** (If demand >0)
  $$ \text{CoverDays} = \frac{\text{UsableStock}}{D}. $$
  E.g. 100 stock with 20/day demand → 5 days cover.

- **Lead-time Demand:**
  For transfer lead time L (in days):
  $$ \text{LeadTimeDemand} = D \times L. $$

- **Replenishment Gap:**
  (units needed by arrival):
  Let SafetyStock = minimal desired cover.

$$ \text{Gap} = \max(0, \text{LeadTimeDemand} + \text{SafetyStock} - (\text{UsableStock} + \text{IncomingBeforeNeedDate})) .$$
E.g., if lead-demand=40, safety=10, usable=15, incoming=0, then Gap = max(0,50-15)=35.

- **Safe Source Surplus:**
  For each source warehouse:
  $$ \text{Surplus} = \text{UsableStock}*{\text{source}} - \text{Reserve}*. $$
  Reserve might be e.g. a fixed safety stock per source to not drop it below critical. If no explicit reserve, we could use safety stock formula.

- **Feasible Transfer Quantity:**
  A transfer from S to D for SKU is limited by:

- Destination's gap (above).
- Source's surplus.
- Transport capacity (if, say, max units per truck).
- Destination capacity (if any).
- Pack-size constraints (if only cases).

- Batch constraints: ensure shelf-life at arrival.
  The algorithm will propose $Q = \min(\text{Gap}_D,\ \text{Surplus}_S,\ \text{CapacityLimits})$ rounding to pack units.

- **Recommendation Ranking Score:**
  To rank suggestions, one possible formula (configurable weights):
  $$ \text{Score} = w_1\frac{\text{Urgency}}{\max\text{Urgency}} + w_2\frac{1}{\text{Cost}} + w_3\frac{\text{RemainingShelf}}{\max\text{Shelf}} - w_4\text{LeadTime}. $$
  Where Urgency may be inverse of CoverDays (smaller cover = higher urgency). We will normalize factors. These w_i are design choices (e.g. emphasize urgent stockouts heavily). In the prototype, we will set weights (e.g. 0.5 urgency, 0.3 cost, 0.2 expiry, etc.) and allow adjustment. The key is transparency: we will explain each recommendation by the factors.

- **Decision Flow (diagram 6):**

- Compute each destination's projected shortage (Gap).
- Identify if Gap > 0 (need exists).
- For each needed SKU-location, find eligible source warehouses with Surplus > 0.
- Check transport route (exists, lead-time meets requirement, cost known).
- Check expiry: estimate shelf-life at arrival (current life – lead-time). If below threshold, disallow.
- If all checks pass, compute feasible quantity (min of need and surplus, adjust pack).
- If quantity >0, propose recommendation; else flag as infeasible.
- Assign score and rank.
- Repeat for next gap or stop when no more feasible moves or sources exhausted.

If data missing (e.g. no route), we mark recommendation as *Rejected (no route)*. If sales history is missing, we skip forecasting for that SKU and mark *no demand data*. In general, any missing data leads to an explanation note and skipping of that potential move.

10. Worked Numerical Example

(All numbers here are illustrative, simulated data.)

Suppose:

- **Warehouse A:** 90 units of SKU X, reserve 30, usable 60.
- **Warehouse B:** 20 units of SKU X, reserve 10, usable 10.
- **Retail North:** Stock 15, forecast demand 20/day, incoming 0, safety stock 10. Lead time from A=2 days, from B=4 days. Transport cost ₹2/unit from either.
- **Retail South:** Stock 40, demand 5/day (low), not urgently needed.

Compute for North:
LeadTimeDemand_A = 20×2 = 40; target = 40+10=50; gap = max(0,50 – 15) = 35 units needed (by day ~Nov. X).
SafeSurplus_A = 60 – 30 = 30 (A must keep 30). Actually, if reserve=30, then usable 30, not enough. If reserve=0, surplus=60. Let reserve=30, so Surplus_A=30.
LeadTimeDemand_B = 20×4 = 80; target = 90; gap still 50; Surplus_B = 10–?= say 0 (if B reserve=10, no spare).

So only A can supply. Feasible quantity = min(35 need, 30 surplus) = 30 (pack units no constraint here).
Check expiry: Assume all have >6 months shelf, OK.
Score this suggestion: Urgency high (cover at North 15/20=0.75 days), low cost (₹60), lead-time 2 days. Put it rank 1.
Now left need = 5, but no more source surplus. So recommendation list: Transfer 30 units of X from A→North (Cost ₹60). Another possible suggestion: *Propose A→South?* No, South doesn't need X (no shortage) – we won't send it there.

Meanwhile, maybe another SKU Y: (not shown here).

Demonstrate a rejected example: If we considered transfer 50 units from A→North (exceeding surplus), the engine would reject: "Qty exceeds A's safe surplus (only 30 avail)." Or if A→North with 30 but lead time was 5 days (arrives after need date), it would reject: "Cannot arrive in time."

This example shows the arithmetic behind days of cover and gap. All intermediate values (15 stock, 20/day demand, 2-day lead) are clearly labeled as simulated inputs. The engine picks the largest feasible transfer for the most urgent gap and provides the breakdown above.

11. Evaluation Methodology

We will compare ORIGIN's recommendations to a **baseline policy** using the same data. Possible baseline: "No proactive transfers; only reorder at safety stock via static rules." For fairness, define baseline as "whenever stock hits reserve at a retailer, do nothing vs. ORIGIN's transfers." We'll simulate both through the planning horizon (e.g. 14 days).

Metrics:

- **Stockout Risk Units:** Units of forecast demand (over period) not covered by stock + incoming before needed (total potential unmet demand). Lower is better.
- **Fill Rate:** (Units fulfilled / units demanded) over period, e.g. 98% vs 90%.
- **Total Transfer Cost:** Sum of (unit cost × quantity) of shipments in scenario.
- **Expiry-Risk Units:** Units likely to expire (stock beyond demand before expiry date). Lower is better.
- **Constraint Violations:** How many recommendations broke rules (should be zero after filter).

- **Forecast Error (if actual demand known):** e.g. Mean Absolute Percent Error between predicted and actual sales (for analysis only).

We will present results for baseline vs ORIGIN: e.g.
Metric	Baseline	ORIGIN	Difference	% Improvement	Notes
Stockout units	100 units	20 units	–80 units	–80%	(lower is better)
Fill rate	85%	98%	+13 pp	+15%	
Transfer cost (₹)	0 ₹	140 ₹	+140 ₹	N/A	cost incurred * vs saved sales
Expiry risk (units)	30	10	–20	–67%	
Violations	N/A	0	0	0%	All ORIGIN recs valid

(These numbers are hypothetical.) We will specify evaluation period and dataset size (e.g. 4 SKUs, 5 locations, 14-day forecast). Limitations: our test is illustrative unless run on real or large data. We avoid claims like "X% improvement in all cases" – results will depend on scenario. The system should be tested on larger or real data later to verify any consistent gains. For now, we show how to compute each metric clearly.

12. Implementation Roadmap

We define an MVP scope and possible extensions:

- **MVP Features (to build):**
- **CSV Upload & Parse:** Import inventory, sales, routes. Basic UI form.
- **Data Validation Checks:** As in Screen 2. Reports missing/inconsistent data.
- **Inventory-Risk Calculations:** Compute demand and gap per SKU/location.
- **Source-Surplus Calculation:** Safe surplus = stock – reserve.
- **Recommendation Generation:** For each urgent need, find best transfer. Include cost, check lead time, expiry. Output raw list.
- **Recommendation UI:** Table of suggestions with details as in Screen 5. Explanation text generation.
- **Comparison Scenario:** Simulate baseline (no transfers or simple rule) vs. ORIGIN results and display metrics.
- **Exportable Results:** Allow export of recommendations CSV or PDF report.

Effort & order:
1. Data model design and CSV import (high priority).
2. Demand forecasting and stock logic (compute cover, gap).
3. Simple recommendation loop (maximize urgent gaps).
4. Frontend displays (tables) for risk analysis and recommendations.
5. Baseline simulation and comparison.
6. UI polish, validation warnings.

- **Optional If Time:**
- More advanced forecasting (e.g. EWMA).
- Interactive maps (using location coordinates).
- Scenario sliders (e.g. vary weights, demand levels).
- Pack-size constraints.
- Use OR-Tools for optimization (if heuristics insufficient).
- Improved expiry modeling (per-batch shelf life).

- Multi-language support (since team has India context).

- **Future (post-hackathon):**

- Real ERP/ERPnext integrations for live data.
- Live driver/vehicle tracking to enforce lead time.
- Cloud deployment with real-time data pipeline.
- Automated purchase order/shipment creation.
- Scaling tests on large real datasets.

Acceptance Criteria: At each stage, the output should match expected calculations (e.g. the risk screen matches manual gap calculations) and the UI displays all required fields. Unit tests for formulas and end-to-end flow will ensure correctness. We will not attempt features beyond MVP unless all core parts work.

13. Technology Stack

- **Frontend:** We may use **React** (for a modern dashboard) or **Streamlit** (faster dev but less customized). React + a charting library (e.g. Chart.js) will allow richer visuals (maps, interactive tables).
- **Backend/Data: Python 3** with **pandas** for CSV processing. Flask or FastAPI can serve API endpoints if needed.
- **Storage:** For prototype, no DB needed – keep data in-memory or simple SQLite for user session.
- **Visualization:** Chart.js or Recharts in React, or Altair/Plotly. For simplicity, we might start with static Chart.js via React.
- **Optimization/Algorithm:** Custom code in Python. If time, the **PuLP** or **OR-Tools** library could solve small LP models (e.g. min-cost flow), but transparency suffers. We will likely implement greedy heuristics as above.
- **Deployment:** Hosted on local machine or basic cloud (e.g. GitHub Codespace or a VM for demo).

Justification: This common web stack balances development speed and flexibility. Python + pandas is ubiquitous for data processing (our team's expertise). React provides a professional UI. We avoid exotic tech to keep focus on logic. We may prototype some screens first in Streamlit to validate logic, then build final UI in React if time.

14. Testing and Edge Cases

We will test for:

- **Missing data:** If sales history is empty, demand=0 or flagged. If route cost missing, recommendation is infeasible. System should not crash.
- **Zero demand or stock:** If demand=0, skip allocation (no need). If stock=0 at all sources, warn "no supply".
- **Pack sizes:** If product is in cases of 10, round transfer to nearest case. If gap is less than pack size, likely skip or send 1 pack.
- **Expired stock:** If batch is already expired, exclude from usable. If shipment arrives after expiry date, skip recommendation.
- **Transport capacity:** If too large quantity (exceeding a truck's max), split into multiple or skip (we can initially ignore capacity or assume large enough).
- **Data errors:** If CSV columns mislabeled, show validation error. If negative inventory, treat as 0 or error.
- **Dynamic changes:** Our UI is not real-time; assume static snapshot upload.

We will write unit tests for calculation functions (cover days, gap, surplus) and integration tests on small synthetic datasets (comparing hand-computed results to code output). All formulas will be double-checked (like the worked example).

15. Assumptions, Risks, and Limitations

- **Data Assumptions:** We assume reasonably accurate sales history (no radical shifts). We assume cost per unit or per tonne is given. We take lead-times as deterministic (no variability).
- **Modeling Assumptions:** We use average-demand forecast; we assume demand is smooth. High seasonality or promotions could break this (future work: advanced forecasting). Safety stock is fixed. We ignore perishability complexity beyond expiry date check.
- **Scope Limitations:** Prototype only handles push allocation; we do not model procurement or production planning. We do not simulate dynamic re-planning if demand changes after scheduling.
- **Integration Limits:** This demo uses CSVs; real systems have APIs/ERPs. The app doesn't connect to real warehouses or carriers.
- **Performance:** We assume small data (hundreds of SKUs, locations); not optimized for big data.
- **Risk:** Missing or wrong assumptions (e.g. ignoring volume capacity) might lead to unrealistic plans. We will clearly label limitations on the Evidence screen.

16. Future Opportunities

Beyond MVP:

- **Machine Learning Forecasting:** Incorporate seasonal indices or ML for demand, tied with live sales data.
- **Optimization Engine:** Replace greedy algorithm with linear/integer programming to maximize fill rate under cost constraint.
- **ERP Integration:** Connect with systems (SAP, ERPNext) to pull live inventory and push orders.
- **Real-Time Tracking:** Update recommendations if delays occur.
- **User Feedback Loop:** Allow managers to tweak weights or provide outcomes, and refine algorithm.
- **Scaling:** Extend to 3+ tiers (multi-tier distribution, international).
- **AI Explanation:** Use LIME/SHAP to further clarify "why this recommendation."

17. Final Summary

We propose **ORIGIN: Distribution Intelligence & Inventory Optimization Engine** to assist FMCG supply chains. By leveraging data on inventory, demand, transport, and expiry, ORIGIN will generate prioritized transfer recommendations to proactively address stock imbalances. Our plan is grounded in supply-chain research and existing system capabilities. We will build a clear, testable prototype showing concrete calculations and results, not just a high-level concept. All claims are based on cited sources or labeled simulations. With this rigorous approach, a hackathon judge (even non-technical) can see *what we did, why it matters, and how it works* from statistics to algorithms.

References

1. Food and Agriculture Organization (FAO), *"Global food losses and food waste – Extent, Causes and Prevention"* (2012) – "roughly one-third of food produced for human consumption is lost or wasted globally, about 1.3 billion tons/year"【34†L13-L21】.

2. Press Information Bureau (PIB), Govt. of India, *"Assessment of Logistics Cost in India"* (2025) – Logistics cost ≈7.97% of GDP (₹24 lakh crore) in FY2023–24【45†L99-L107】.
3. Oracle NetSuite Resource (2020) – "In the U.S. retail food industry alone, stockouts cost $15–$20 billion/year (≈3% of sales)"【43†L7-L9】; global retail stockout rate ~8%【43†L31-L39】.
4. Fishbowl Inventory blog (2026) – "Most companies aim to keep inventory carrying cost between 20% and 30% of inventory value"【51†L799-L807】 (ISM cited).
5. Zoho Inventory product page – Multi-warehouse and stock transfer capabilities【63†L235-L244】; batch/serial tracking【64†L1-L4】.
6. Odoo (via partner site) – Multi-location management, advanced reordering (min/max) rules, expiry/lot tracking (FEFO)【77†L134-L142】【77†L144-L146】.
7. SAP Integrated Business Planning (official) – Combines demand, supply, inventory, S&OP planning; features inventory optimization【98†L35-L44】【98†L208-L218】.
8. SoftwareReviews (Blue Yonder Luminate) – Planning features including demand planning, inventory optimization, allocation & replenishment【88†L72-L82】.
9. Kinaxis (Wikipedia) – Platform supports planning across demand, supply, inventory, S&OP 【95†L151-L159】.

(URL links to sources are provided in the citation markers above.)
